

01MIAR - Actividad Whitepapers

Introducción:

- Para abordar un proyecto de Inteligencia Artificial es necesario disponer del conocimiento suficiente tanto a nivel teórico como técnico para garantizar el éxito en la consecución de los objetivos. La fase de investigación sobre la temática de un proyecto es indispensable para conocer nuevos enfoques o afianzar y dar sustento a las acciones que se llevarán a cabo en los trabajos. Los artículos científicos son la referencia más utilizada para investigar sobre un ámbito de conocimiento ya que, antes de su publicación, son aceptados en un proceso llamado "revisión por pares".

Objetivo

- Investigar sobre el estado actual del conocimiento sobre procesamiento de datos en trabajos relacionados con Inteligencia Artificial, en concreto, con el uso de Python y librerías.
- El propósito de esta actividad es conocer e indagar en los contenidos definidos en la asignatura Python para la Inteligencia Artificial mediante artículos científicos, siendo estos el marco del conocimiento a investigar.

Metodología

- Buscar y seleccionar un artículo científico relacionado con la asignatura, es decir, Python o IA.
 - Google Académico (Scholar) es un buscador muy recomendado, aporta una primera idea sobre qué hay publicado y en qué revistas científicas.
 - También se aceptarán artículos publicados en conferencias (proceedings) o "preprints" en [arXiv](#) siempre y cuando tengan un formato académico.
 - La fecha de publicación del artículo debe ser actual, no se aceptarán artículos previos al 2024.
- Crear un notebook de Python que servirá como entrega de la actividad (se entregará el fichero .ipynb). En esta entrega se debe:
 - Referenciar adecuadamente este artículo con su estilo de preferencia

(IEEE, Harvard, Science).

- Describir por qué has seleccionado este artículo (trabajo, estudios, hobbies, otros intereses...). Explicar que quieres aprender y su usabilidad.
- Resumir el contenido del artículo.
- Seleccionar un ejemplo de implementación con Python. El código puede ser extraído directamente del artículo.
- Explicar el código implementado.
- Concluir con un párrafo de reflexión. ¿Qué has aprendido?
- Se recomienda:
 - Formatear bien los apartados propuestos con celdas Markdown o de código según corresponda.
 - Mantener una estructura de código limpia, con comentarios oportunos y tomando en cuenta la secuencia en la que se ejecuta el código.
 - Utilizar un formato adecuado, si aplica, para referencias complementarias.
 - Sintetizar la información, aunque no hay límite de palabras.
 - Mantener una buena ortografía.

IMPORTANTE:

- El uso de inteligencia artificial debe estar acorde a la normativa descrita en la guía docente.
- No está permitido compartir los resultados ni el código en ninguno de los foros.

Evaluación

- Esta actividad corresponde con el 10% de la nota final de la asignatura.

Entrega:

- Un solo documento en formato notebook: .ipynb
- Las entregas sólo serán válidas si se realizan a través del campus virtual: Actividades Formativas, Actividad WhitePapers.

Fechas Límite

- Convocatoria 1 - 01/06/2026 - 08.00 CET
- Convocatoria 2 - 21/09/2026 - 08.00 CET

Rúbrica

Apartados	Criterio 1	Criterio 2	Criterio 3
Estilo (20%)	Redacción y ortografía	Formato correcto del notebook	Formato de las referencias
Artículo (20%)	Relación con la asignatura	Publicación actual	Relevancia académica
Contenidos (60%)	Justificación y reflexión	Análisis del artículo	Ejemplo en Python

Solución de la Actividad

Búsqueda y selección de un artículo científico relacionado con la asignatura

Considerando mi experiencia en programación python y otras disciplinas de mi interés, pensé que podría trabajar en un análisis de análisis financiero de factores, en la que pudiera utilizar los conceptos aprendidos en la materia.

Entonces realicé en la plataforma Google Scholar la siguiente búsqueda: "python machine learning financial factor analysis". Entre los resultados obtenidos, me resultó interesante el capítulo 1 del libro **Machine Learning in Business Finance Using Python**, "Financial Data Correlations". Verifiqué que la fecha de publicación de este libro es de Septiembre de 2025, lo cual cumple con la consigna de esta actividad de que el whitepaper sobre el que se haga el trabajo tenga fecha de publicación actual, no previos al 2024. También verifiqué que el trabajo elegido realiza procesamiento de datos en un contexto relacionado con Inteligencia Artificial, que es requisito para esta actividad. Si bien el artículo elegido no es de IA "pura" o modelado de machine learning, en el artículo aparecen tareas típicas de preprocesado de datos, carga de CSV, limpieza, transformación de columnas, cálculo de retornos y optimización de carteras, que son habilidades muy usadas en proyectos de IA y data science y también fueron estudiadas en el transcurso de las clases de la materia. Por último, la elección de este artículo científico fue validado con el Profesor del Curso.

El trabajo elegido es este según su referencia Harvard: Lim, K.G. (2025) **Machine Learning in Business Finance Using Python**. Singapore: World Scientific Publishing.

Mi interés en elegir este artículo fue utilizar tanto los conocimientos de python adquiridos en la materia como los conocimientos que yo adquiriré en una Maestría

en Finanzas que he realizado en el pasado. Me gustaría aprender la aplicación de las librerías `numpy` y `pandas` a esta área de mi interés profesional como es el análisis de factores financieros y de qué manera se pueden usar estas librerías.

Resumen del artículo

Resumen: Capítulo 1 — Correlaciones en Datos Financieros

Idea Central

La idea de las correlaciones de datos financieros (retornos de los activos en un mismo período de tiempo) es la relación fundamental que gobierna la mayoría de las inversiones financieras. Si cada serie de datos financieros, como las tasas de retornos de las acciones, es probabilísticamente independiente de las demás acciones, entonces no existen factores de riesgo sistemáticos que afecten a todas las acciones; cada acción se mueve según su propio riesgo independiente. Esto último no refleja las interrelaciones reales entre el mercado bursátil y el sector corporativo. Las empresas cotizadas sí comparten riesgos comunes, como cuando pertenecen al mismo sector afectado por un cambio sectorial o cuando se ven afectadas por el coste de los fondos al variar los tipos de interés del mercado. Cuando la rentabilidad de las acciones se mueve conjuntamente o presenta correlaciones distintas de cero, pueden combinarse en una cartera de forma óptima para maximizar el beneficio del inversor.

1. Revisión de herramientas Python

El capítulo arranca con un repaso práctico de **NumPy** (operaciones matriciales y arrays) y **Pandas** (manipulación de datos tabulares), que se usan a lo largo de todo el análisis.

2. Diversificación de Portafolios (Markowitz)

Se presentan tres problemas de optimización bajo restricciones de no-venta en corto:

- **Frontera eficiente de Markowitz:** representa el límite de portafolios de acciones riesgosas que tienen el menor valor de varianza sobre los retornos dado el valor esperado de retorno para esa combinación de activos y que el retorno esperado sea mayor o igual al retorno del MVP (Minimum variance portfolio). El gráfico de retornos medios versus la volatilidad (o desvío estándar de los retornos, o riesgo) cuando no hay restricciones de ventas en corto tiene la forma de una hipérbola, y es usualmente llamado el portafolio de la frontera eficiente.
- **Máximo Sharpe Ratio:** maximizar el retorno esperado por unidad de riesgo.

El ejemplo práctico usa 7 empresas de e-commerce (Alibaba(BABA.US), Amazon(AMZN.US), eBay (EBAY.US), Rakuten (4755.jp), Wayfair (W.US), Zalando (zal.de), JD.com (JD.US)), con datos descargados del sitio <https://stooq.com> del período 2017–2021. Los retornos se calculan como logaritmos continuamente compuestos sobre precios ajustados por dividendos y splits. NOTA: he eliminado la empresa Suning incluida en el libro por no haber podido descargar los retornos de esta empresa japonesa.

3. Portafolios Multi-sector

Se amplía el universo a 24 acciones en tres sectores (e-commerce, semiconductores, alimentos). Se demuestra que **la frontera eficiente combinada domina a cada sector por separado**: para igual retorno esperado, la volatilidad es menor — evidencia directa del beneficio de la diversificación. Sin embargo, los pesos óptimos calculados en el conjunto de entrenamiento (2017–2020) **no se transfieren bien** al conjunto de test (2021), porque los retornos futuros difieren de los históricos. Esto ilustra el principal talón de Aquiles de la optimización clásica.

4. Denoising de la Matriz de Correlación

El problema: con N activos y T observaciones finitas, la matriz de correlación muestral es una variable aleatoria, no la verdadera matriz poblacional. El **Teorema de Marchenko–Pastur** establece que, si los datos fueran puro ruido, los autovalores de la matriz de correlación seguirían una distribución teórica acotada entre L y U . Los autovalores **por encima de $U \approx 1,3$** (para $N=24$, $T=889$) representan señales genuinas del mercado; los que caen por debajo son ruido. **Método de denoising** (constant residual eigenvalue): reemplazar los autovalores menores que U por su promedio (0,5719), preservando la varianza total, y reconstruir la matriz de covarianza. En este caso particular el beneficio fue marginal, probablemente porque el error en la estimación de los retornos esperados domina sobre el error en la covarianza.

5. Predictor Prospectivo Más Preciso

La sección más importante conceptualmente: si se sustituyen los retornos históricos medios por **predicciones más precisas de los retornos futuros**, el portafolio out-of-sample se acerca considerablemente a la frontera eficiente in-sample. Esto demuestra que la calidad de la predicción de retornos es más determinante para el desempeño real que la diversificación en sí misma.

6. Conclusiones

Aspecto	Limitación	Posible mejora con ML
Retornos esperados	Se usan medias históricas, que no predicen bien el futuro	Redes neuronales, modelos de factores
Covarianza	Estimación ruidosa con muestras finitas	Denoising, GARCH para varianzas
Validación	Back-testing sobre ventanas rodantes	Estrategias con significancia estadística de ganancia

Librerías utilizadas en el capítulo: NumPy, Pandas, Matplotlib, Seaborn, SciPy (optimize), scikit-learn (KernelDensity).

Selección de un ejemplo de implementación con Python

He seleccionado el código del artículo que corresponde al punto **1.2 Worked example: Data**, que corresponde al punto **2. Diversificación de portfolios (Markowitz)** de este documento.

Para poder obtener los retornos de las empresas seleccionadas, he exportado del sitio stooq.com los retornos diarios entre el 1 de enero de 2017 al 31 de diciembre de 2021 de las empresas con el siguiente criterio:

Empresa	Ticker	Nombre del Archivo con retornos diarios
Alibaba	BABA.US	datafiles/baba_us_d.csv
Amazon	AMZN.US	datafiles/amzn_us_d.csv
Ebay	EBAY.US	datafiles/ebay_us_d.csv
Rakuten	4755.jp	datafiles/4755_jp_d.csv
Wayfair	W.US	datafiles/w_us_d.csv
Zalando	ZAL.DE	datafiles/zal_de_d.csv
JD.com	JD.US	datafiles/jd_us_d.csv

Ejemplo de la url utilizada para la empres **Alibaba**: <https://stooq.com/q/d/?f=20170101&t=20211231&s=BABA.US&c=0>

Requisitos previos: Antes de ejecutar este notebook, es necesario instalar los paquetes que el notebook requiere en el ambiente python donde se ejecutará. Esto se hace con los siguientes comandos:

```
In [88]: !pip install numpy
!pip install pandas
!pip install matplotlib
!pip install scipy
```

Requirement already satisfied: numpy in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (2.3.5)

Requirement already satisfied: pandas in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (2.3.3)

Requirement already satisfied: numpy>=1.26.0 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from pandas) (2.3.5)

Requirement already satisfied: python-dateutil>=2.8.2 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from pandas) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from pandas) (2026.2)

Requirement already satisfied: tzdata>=2022.7 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from pandas) (2026.2)

Requirement already satisfied: six>=1.5 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from python-dateutil>=2.8.2->pandas) (1.17.0)

Requirement already satisfied: matplotlib in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (3.10.9)

Requirement already satisfied: contourpy>=1.0.1 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (1.3.3)

Requirement already satisfied: cycler>=0.10 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (0.12.1)

Requirement already satisfied: fonttools>=4.22.0 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (4.62.1)

Requirement already satisfied: kiwisolver>=1.3.1 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (1.5.0)

Requirement already satisfied: numpy>=1.23 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (2.3.5)

Requirement already satisfied: packaging>=20.0 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (26.2)

Requirement already satisfied: pillow>=8 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (12.2.0)

Requirement already satisfied: pyparsing>=3 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (3.3.2)

Requirement already satisfied: python-dateutil>=2.7 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from matplotlib) (2.9.0.post0)

Requirement already satisfied: six>=1.5 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)

Requirement already satisfied: scipy in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (1.17.1)
 Requirement already satisfied: numpy<2.7,>=1.26.4 in /Users/josemaestro/anaconda3/envs/VIU-PythonparaAI/lib/python3.13/site-packages (from scipy) (2.3.5)

A continuación, se detalla el código elegido para la actividad de Whitepaper:

```
In [89]: ### This module computes returns and mean-var frontier given each
### "expected" return based on averaged realized return
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
```

```
In [90]: import pandas as pd

github_directory = "https://raw.githubusercontent.com/jmaestro-dev/mas
github_datafiles_directory = os.path.join(github_directory, "datafiles

df = pd.read_csv(os.path.join(github_datafiles_directory, "baba_us_d.c
df = df.sort_values(df.columns[0])
df.to_csv("Alibaba_USD_clean.csv", index=False)
df.head()
```

```
Out[90]:
```

	Date	Open	High	Low	Close	Volume
0	2017-01-03	89.000	89.00	88.08	88.60	8787301
1	2017-01-04	88.985	90.89	88.58	90.51	11462690
2	2017-01-05	91.910	94.81	91.64	94.37	16787882
3	2017-01-06	94.400	94.50	93.00	93.89	7597313
4	2017-01-09	94.160	95.65	93.31	94.72	10817809

```
In [91]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1259 entries, 0 to 1258
Data columns (total 6 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Date    1259 non-null     object
1    Open    1259 non-null     float64
2    High    1259 non-null     float64
3    Low     1259 non-null     float64
4    Close   1259 non-null     float64
5    Volume  1259 non-null     int64
dtypes: float64(4), int64(1), object(1)
memory usage: 59.1+ KB
```



```
In [92]: df[["year", "month", "day"]] = df["Date"].str.split("-", expand=True)
```

```
In [93]: ### Above splits the Date entry into 3 separate columns
print("\nNew DataFrame:")
print(df)
```

New DataFrame:

	Date	Open	High	Low	Close	Volume	year	month	day
0	2017-01-03	89.000	89.00	88.0800	88.60	8787301	2017	01	03
1	2017-01-04	88.985	90.89	88.5800	90.51	11462690	2017	01	04
2	2017-01-05	91.910	94.81	91.6400	94.37	16787882	2017	01	05
3	2017-01-06	94.400	94.50	93.0000	93.89	7597313	2017	01	06
4	2017-01-09	94.160	95.65	93.3100	94.72	10817809	2017	01	09
...	...	...	...	...	...	...	...	...	...
1254	2021-12-27	117.800	121.80	116.1021	116.59	17864372	2021	12	27
1255	2021-12-28	116.260	116.42	114.4700	114.80	17052102	2021	12	28
1256	2021-12-29	113.800	114.20	110.3800	112.09	21100463	2021	12	29
1257	2021-12-30	112.630	124.30	112.3000	122.99	45437131	2021	12	30
1258	2021-12-31	121.230	122.91	118.5900	118.79	24287047	2021	12	31

[1259 rows x 9 columns]

```
In [94]: cols = df.select_dtypes(exclude=['float']).columns
df[cols] = df[cols].apply(pd.to_numeric, downcast='float', errors='coerce')
### Above selects non-float type as cols and converts the Date, day, month, and year from objects to float#
### errors = 'coerce' sets invalid parsing as NaN such as Date column
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1259 entries, 0 to 1258
Data columns (total 9 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        0 non-null      float32
1   Open        1259 non-null   float64
2   High        1259 non-null   float64
3   Low         1259 non-null   float64
4   Close       1259 non-null   float64
5   Volume      1259 non-null   float64
6   year        1259 non-null   float32
7   month       1259 non-null   float32
8   day         1259 non-null   float32
dtypes: float32(4), float64(5)
memory usage: 69.0 KB
```

```
In [95]: dfAlibaba = df[(df['year']>=2017)&(df['year']<=2021)]
dfAlibaba = dfAlibaba.drop('Date',axis=1)
### 1 is the axis number (0 for rows and 1 for columns)
### If we do not drop the Date column, Date will be shown as NaN
### Includes only rows where years are between 2017 and 2021 inclusive
### Next remove all columns except keeping Adj Close (closing price in
### dividends and split effects) and day, month, year
dfAlibaba = dfAlibaba[['Close','day','month','year']]
print(dfAlibaba)
```

```
      Close  day  month  year
0      88.60  3.0   1.0  2017.0
1      90.51  4.0   1.0  2017.0
2      94.37  5.0   1.0  2017.0
3      93.89  6.0   1.0  2017.0
4      94.72  9.0   1.0  2017.0
...      ...  ...   ...   ...
1254  116.59  27.0  12.0  2021.0
1255  114.80  28.0  12.0  2021.0
1256  112.09  29.0  12.0  2021.0
1257  122.99  30.0  12.0  2021.0
1258  118.79  31.0  12.0  2021.0
```

[1259 rows x 4 columns]

```
In [96]: dfAmazon = pd.read_csv(os.path.join(github_datafiles_directory, "amzn_
dfAmazon[['year', 'month', 'day']] = dfAmazon["Date"].str.split("-", e
cols = dfAmazon.select_dtypes(exclude=['float']).columns
dfAmazon[cols] = dfAmazon[cols].apply(pd.to_numeric, downcast='float',
dfAmazon = dfAmazon[(dfAmazon['year']>=2017)&(dfAmazon['year']<=2021)]
dfAmazon = dfAmazon.drop('Date',axis=1)
dfAmazon = dfAmazon[['Close','day','month','year']]
dfAmazon.head()
```

Out [96]:

	Close	day	month	year
0	37.6835	3.0	1.0	2017.0
1	37.8590	4.0	1.0	2017.0
2	39.0225	5.0	1.0	2017.0
3	39.7995	6.0	1.0	2017.0
4	39.8460	9.0	1.0	2017.0

In [97]: df.head()

Out [97]:

	Date	Open	High	Low	Close	Volume	year	month	day
0	NaN	89.000	89.00	88.08	88.60	8787301.0	2017.0	1.0	3.0
1	NaN	88.985	90.89	88.58	90.51	11462690.0	2017.0	1.0	4.0
2	NaN	91.910	94.81	91.64	94.37	16787882.0	2017.0	1.0	5.0
3	NaN	94.400	94.50	93.00	93.89	7597313.0	2017.0	1.0	6.0
4	NaN	94.160	95.65	93.31	94.72	10817809.0	2017.0	1.0	9.0

```
In [98]: dfEbay = pd.read_csv(os.path.join(github_datafiles_directory, "ebay_us
dfEbay[["year", "month", "day"]] = dfEbay["Date"].str.split("-", expand
cols = dfEbay.select_dtypes(exclude=['float']).columns
dfEbay[cols] = dfEbay[cols].apply(pd.to_numeric, downcast='float', err
dfEbay = dfEbay[(dfEbay['year']>=2017)&(dfEbay['year']<=2021)]
dfEbay = dfEbay.drop('Date',axis=1)
dfEbay = dfEbay[['Close','year','month','day']]
dfEbay.head()
```

Out [98]:

	Close	year	month	day
0	28.1317	2017.0	1.0	3.0
1	28.0548	2017.0	1.0	4.0
2	28.2913	2017.0	1.0	5.0
3	29.2720	2017.0	1.0	6.0
4	28.9882	2017.0	1.0	9.0

```
In [99]: dfRak = pd.read_csv(os.path.join(github_datafiles_directory, "4755_jp_
dfRak[["year", "month", "day"]] = dfRak["Date"].str.split("-", expand
cols = dfRak.select_dtypes(exclude=['float']).columns
dfRak[cols] = dfRak[cols].apply(pd.to_numeric, downcast='float', error
dfRak = dfRak[(dfRak['year']>=2017)&(dfRak['year']<=2021)]
dfRak = dfRak.drop('Date',axis=1)
```

```
dfRak = dfRak[['Close', 'year', 'month', 'day']]
dfRak.head()
```

Out[99]:

	Close	year	month	day
0	1170.0	2017.0	1.0	4.0
1	1180.0	2017.0	1.0	5.0
2	1221.5	2017.0	1.0	6.0
3	1213.5	2017.0	1.0	10.0
4	1216.5	2017.0	1.0	11.0

In [100...]

```
dfWay = pd.read_csv(os.path.join(github_datafiles_directory, "w_us_d.c
dfWay[["year", "month", "day"]] = dfWay["Date"].str.split("-", expand
cols = dfWay.select_dtypes(exclude=['float']).columns
dfWay[cols] = dfWay[cols].apply(pd.to_numeric, downcast='float', error
dfWay= dfWay[(dfWay['year']>=2017)&(dfWay['year']<=2021)]
dfWay = dfWay.drop('Date',axis=1)
dfWay = dfWay[['Close', 'year', 'month', 'day']]
dfWay.head()
```

Out[100...]

	Close	year	month	day
0	35.18	2017.0	1.0	3.0
1	36.48	2017.0	1.0	4.0
2	37.17	2017.0	1.0	5.0
3	37.36	2017.0	1.0	6.0
4	38.05	2017.0	1.0	9.0

In [101...]

```
dfZal = pd.read_csv(os.path.join(github_datafiles_directory, "zal_de_d
dfZal[["year", "month", "day"]] = dfZal["Date"].str.split("-", expand
cols = dfZal.select_dtypes(exclude=['float']).columns
dfZal[cols] = dfZal[cols].apply(pd.to_numeric, downcast='float', error
dfZal= dfZal[(dfZal['year']>=2017)&(dfZal['year']<=2021)]
dfZal = dfZal.drop('Date',axis=1)
dfZal = dfZal[['Close', 'year', 'month', 'day']]
dfZal.head()
```

Out [101...

	Close	year	month	day
0	36.905	2017.0	1.0	2.0
1	37.315	2017.0	1.0	3.0
2	35.920	2017.0	1.0	4.0
3	36.765	2017.0	1.0	5.0
4	37.350	2017.0	1.0	6.0

```
In [102... dfJD = pd.read_csv(os.path.join(github_datafiles_directory, "jd_us_d.c
dfJD[["year", "month", "day"]] = dfJD["Date"].str.split("-", expand =
cols = dfJD.select_dtypes(exclude=['float']).columns
dfJD[cols] = dfJD[cols].apply(pd.to_numeric, downcast='float', errors=
dfJD= dfJD[(dfJD['year']>=2017)&(dfJD['year']<=2021)]
dfJD = dfJD.drop('Date',axis=1)
dfJD = dfJD[["Close","year","month","day"]]
dfJD.head()
```

Out [102...

	Close	year	month	day
0	25.1947	2017.0	1.0	3.0
1	25.2239	2017.0	1.0	4.0
2	25.6630	2017.0	1.0	5.0
3	25.6338	2017.0	1.0	6.0
4	25.6240	2017.0	1.0	9.0

```
In [103... ### merging all 8 data files by same day, month, year -- those that do
dfmerge = pd.merge(dfAlibaba,dfAmazon,on = ['day', 'month','year'])
dfmerge.columns=['Alibaba','day','month','year','Amazon'] ### Rename t
dfmerge = pd.merge(dfmerge,dfEbay,on = ['day', 'month','year'])
dfmerge.columns=['Alibaba','day','month','year','Amazon','Ebay']
dfmerge = pd.merge(dfmerge,dfRak, on = ['day', 'month','year'])
dfmerge.columns=['Alibaba','day','month','year','Amazon','Ebay','Rakut
dfmerge = pd.merge(dfmerge,dfWay,on = ['day', 'month','year'])
dfmerge.columns=['Alibaba','day', 'month','year','Amazon','Ebay','Raku
dfmerge = pd.merge(dfmerge,dfZal,on = ['day', 'month','year'])
dfmerge.columns=['Alibaba','day', 'month','year','Amazon','Ebay','Raku
dfmerge = pd.merge(dfmerge,dfJD,on = ['day', 'month','year'])
dfmerge.columns=['Alibaba','day','month','year','Amazon','Ebay','Rakut
```

```
In [104... print(dfmerge)
```

	Alibaba	day	month	year	Amazon	Ebay	RakutenP	Wayfai
r \								
0	90.51	4.0	1.0	2017.0	37.8590	28.0548	1170.0	36.4
8								
1	94.37	5.0	1.0	2017.0	39.0225	28.2913	1180.0	37.1
7								
2	93.89	6.0	1.0	2017.0	39.7995	29.2720	1221.5	37.3
6								
3	96.75	10.0	1.0	2017.0	39.7950	28.5159	1213.5	41.0
4								
4	96.94	11.0	1.0	2017.0	39.9510	28.6688	1216.5	40.8
5								
...	...	...	...	...	...	...	...	...
...								
1153	118.66	23.0	12.0	2021.0	171.0680	63.6907	1124.0	204.3
7								
1154	116.59	27.0	12.0	2021.0	169.6700	64.4465	1135.0	198.8
8								
1155	114.80	28.0	12.0	2021.0	170.6610	64.8587	1150.0	192.8
6								
1156	112.09	29.0	12.0	2021.0	169.2010	65.2321	1164.0	191.7
2								
1157	122.99	30.0	12.0	2021.0	168.6440	65.5455	1154.0	192.8
1								
	Zalando	JDcomp						
0	35.920	25.2239						
1	36.765	25.6630						
2	37.350	25.6338						
3	37.240	26.2192						
4	38.145	26.1217						
...	...	...						
1153	70.520	66.9874						
1154	70.460	66.0701						
1155	70.800	64.2747						
1156	70.700	64.0405						
1157	71.140	68.6950						

[1158 rows x 10 columns]

```
In [105... ### Now delete columns day, month, year; but before that keep day,
### month, year separately for later concatenation
dy=dfmerge["day"]
mth=dfmerge["month"]
yr=dfmerge["year"]
del(dfmerge['day'],dfmerge['month'],dfmerge['year'])
print(dfmerge)
```

	Alibaba	Amazon	Ebay	RakutenP	Wayfair	Zalando	JDcomp
0	90.51	37.8590	28.0548	1170.0	36.48	35.920	25.2239
1	94.37	39.0225	28.2913	1180.0	37.17	36.765	25.6630
2	93.89	39.7995	29.2720	1221.5	37.36	37.350	25.6338
3	96.75	39.7950	28.5159	1213.5	41.04	37.240	26.2192
4	96.94	39.9510	28.6688	1216.5	40.85	38.145	26.1217
...	...	...	...	...	...	...	...
1153	118.66	171.0680	63.6907	1124.0	204.37	70.520	66.9874
1154	116.59	169.6700	64.4465	1135.0	198.88	70.460	66.0701
1155	114.80	170.6610	64.8587	1150.0	192.86	70.800	64.2747
1156	112.09	169.2010	65.2321	1164.0	191.72	70.700	64.0405
1157	122.99	168.6440	65.5455	1154.0	192.81	71.140	68.6950

[1158 rows x 7 columns]

```
In [106... dfret=dfmerge.pct_change().apply(lambda x: np.log(1+x))
### Lambda is an alternative way of defining function inline using a s
### python code. percent change = x.
### Pandas -- Computes the percentage change from the immediately prev
### Note the time series is in ascending order of time (ie later rows
dfret=dfret.iloc[1:len(dfret.index),:]
### Remove NaN values at the first row. Note iloc[0:,...,:] indicates
dfret.columns=['AlibabaP', 'AmazonP', 'EbayP', 'RakutenP', 'WayfairP', 'Zal
### Rename the columns
```

```
In [107... print(dfret)
```

	AlibabaP	AmazonP	EbayP	RakutenP	WayfairP	ZalandoP	JDc
omP							
1	0.041763	0.030270	0.008395	0.008511	0.018738	0.023252	0.017
258							
2	-0.005099	0.019716	0.034077	0.034565	0.005099	0.015787	-0.001
138							
3	0.030006	-0.000113	-0.026170	-0.006571	0.093947	-0.002949	0.022
580							
4	0.001962	0.003912	0.005348	0.002469	-0.004640	0.024011	-0.003
726							
5	-0.010786	0.018132	-0.001994	-0.029616	-0.011820	0.013281	-0.005
998							
...	...	...	...	...	...	...	...
...							
1153	0.007189	0.000181	0.014746	0.005352	0.002842	0.007973	-0.071
660							
1154	-0.017599	-0.008206	0.011797	0.009739	-0.027230	-0.000851	-0.013
788							
1155	-0.015472	0.005824	0.006376	0.013129	-0.030737	0.004814	-0.027
550							
1156	-0.023889	-0.008592	0.005741	0.012100	-0.005929	-0.001413	-0.003
650							
1157	0.092801	-0.003297	0.004793	-0.008628	0.005669	0.006204	0.070
161							

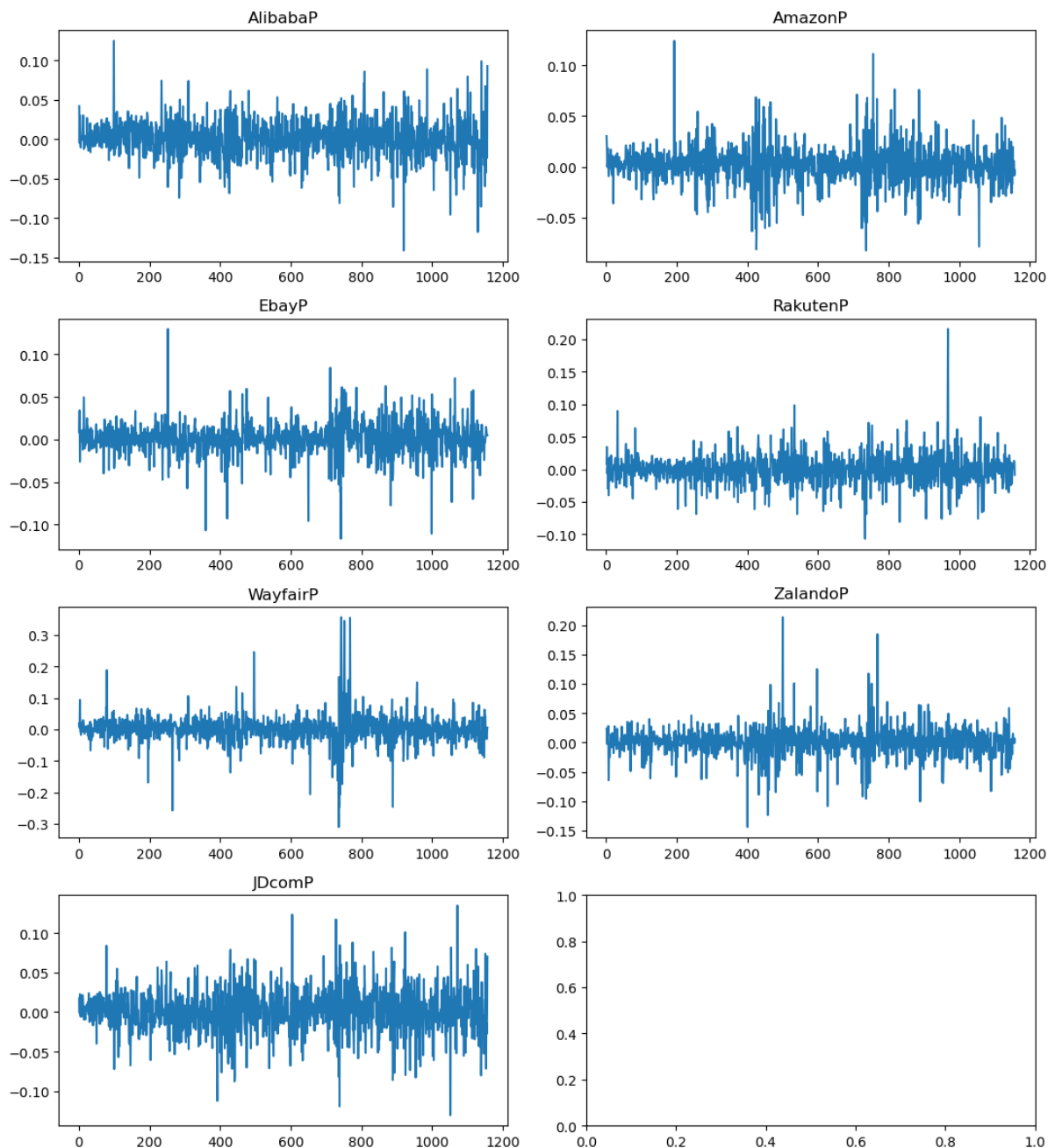
[1157 rows x 7 columns]

In [108... *# He modificado aquí el código propuesto en el libro, utilizando direct*
Esta versión de Matplotlib es más sencilla y evita las reglas de man
Seaborn añade estilos y convenciones de trazado, mientras que Matplo

```
fig, axs = plt.subplots(ncols=2, nrows=4, figsize=(11, 12))
axs = axs.flatten()

for i, (k, v) in enumerate(dfret.items()):
    axs[i].plot(v.index, v.values)
    axs[i].set_title(k)

plt.tight_layout()
plt.show()
```



In [109... *# Finding the simple covariance matrix from a series of returns*


```
covar = dfret.cov()
print(covar)
```

	AlibabaP	AmazonP	EbayP	RakutenP	WayfairP	ZalandoP
JDcomP						
AlibabaP	0.000567	0.000205	0.000138	0.000041	0.000306	0.000126
0.000426						
AmazonP	0.000205	0.000366	0.000133	0.000019	0.000305	0.000128
0.000232						
EbayP	0.000138	0.000133	0.000385	0.000035	0.000276	0.000123
0.000156						
RakutenP	0.000041	0.000019	0.000035	0.000524	0.000114	0.000061
0.000040						
WayfairP	0.000306	0.000305	0.000276	0.000114	0.002086	0.000378
0.000364						
ZalandoP	0.000126	0.000128	0.000123	0.000061	0.000378	0.000623
0.000189						
JDcomP	0.000426	0.000232	0.000156	0.000040	0.000364	0.000189
0.000774						

```
In [110... # Finding the simple correlation matrix from a series of returns
corr_matrix = dfret.corr()
print(corr_matrix)
```

	AlibabaP	AmazonP	EbayP	RakutenP	WayfairP	ZalandoP
JDcomP						
AlibabaP	1.000000	0.451313	0.296381	0.075337	0.281793	0.211414
0.642883						
AmazonP	0.451313	1.000000	0.353781	0.044433	0.349527	0.267832
0.436737						
EbayP	0.296381	0.353781	1.000000	0.078191	0.308661	0.251624
0.286459						
RakutenP	0.075337	0.044433	0.078191	1.000000	0.108846	0.106364
0.062423						
WayfairP	0.281793	0.349527	0.308661	0.108846	1.000000	0.331254
0.286396						
ZalandoP	0.211414	0.267832	0.251624	0.106364	0.331254	1.000000
0.272256						
JDcomP	0.642883	0.436737	0.286459	0.062423	0.286396	0.272256
1.000000						

```
In [111... count = np.isinf(dfret).values.sum()
print(count)
```

0

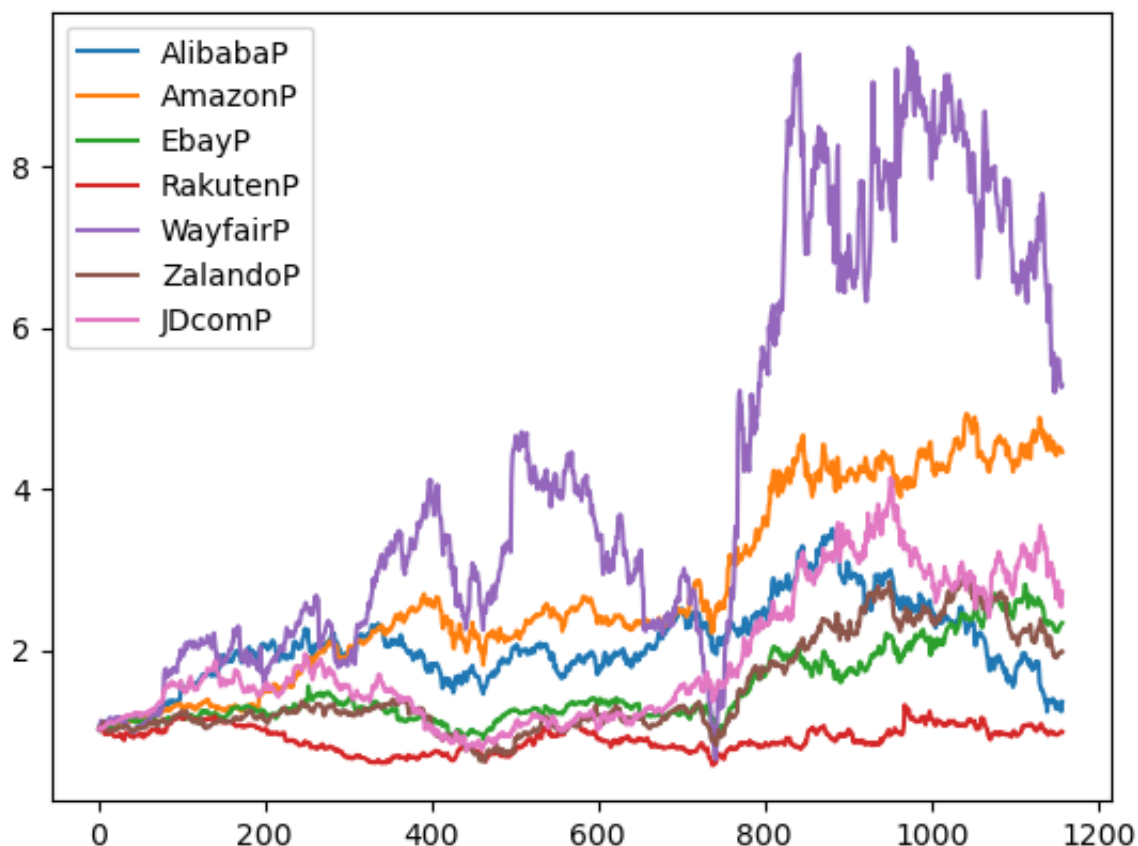
```
In [112... dfcumuret = (np.exp(dfret)).cumprod()
print(dfcumuret)
```

	AlibabaP	AmazonP	EbayP	RakutenP	WayfairP	ZalandoP	JDc
omP							
1	1.042647	1.030732	1.008430	1.008547	1.018914	1.023524	1.017
408							
2	1.037344	1.051256	1.043387	1.044017	1.024123	1.039811	1.016
250							
3	1.068943	1.051137	1.016436	1.037179	1.125000	1.036748	1.039
459							
4	1.071042	1.055258	1.021886	1.039744	1.119792	1.061943	1.035
593							
5	1.059551	1.074566	1.019850	1.009402	1.106634	1.076141	1.029
401							
...	...	...	...	...	...	...	
...							
1153	1.311015	4.518556	2.270225	0.960684	5.602248	1.963252	2.655
711							
1154	1.288145	4.481629	2.297165	0.970085	5.451754	1.961581	2.619
345							
1155	1.268368	4.507805	2.311858	0.982906	5.286732	1.971047	2.548
167							
1156	1.238427	4.469241	2.325167	0.994872	5.255482	1.968263	2.538
882							
1157	1.358855	4.454529	2.336338	0.986325	5.285362	1.980512	2.723
409							

[1157 rows x 7 columns]

```
In [113... fig = plt.figure()
(np.exp(dfret)).cumprod().plot()
plt.show()
```

<Figure size 640x480 with 0 Axes>



```
In [114... import scipy
from scipy.optimize import Bounds
bounds = Bounds(0, 1)

from scipy.optimize import LinearConstraint
linear_constraint = LinearConstraint(np.ones((dfret.shape[1],), dtype=

covar = dfret.cov()
r = np.mean(dfret, axis=0) * 252

def ret(r, w):
    return r.dot(w)

def vol(w, covar):
    return np.sqrt(np.dot(w, np.dot(w, covar)) * 252)

def sharpe(ret, vol):
    return ret / vol

from scipy.optimize import minimize
weights = np.ones(dfret.shape[1])
x0 = weights / np.sum(weights)

portfstderr = lambda w: np.sqrt(np.dot(w, np.dot(w, covar)) * 252)
res1 = minimize(portfstderr, x0, method='trust-constr', constraints=li

w_min = res1.x
```

```
np.set_printoptions(suppress = True, precision=3)
print(w_min)
print('return: % .4f'% (ret(r,w_min)), 'risk: % .4f'% vol(w_min,covar))

[0.096 0.259 0.25  0.273 0.    0.122 0.    ]
return:  0.1542 risk:  0.2060
```

```
In [115... ### Define 1/Sharpe_ratio as invSharpe
invSharpe = lambda w: np.sqrt(np.dot(w,np.dot(w,covar))*252)/r.dot(w)
res2 = minimize(invSharpe,x0,method='trust-constr',constraints = linearConstraint)
### Objective function is invSharpe -- inverse of Sharpe ratio
### These are the weights of the stocks in the portfolio with the high Sharpe ratio
### -- call the weight vector w_Sharpe

w_Sharpe = res2.x
### constraint means unit vector .dot(w) = 1; minimize chooses wts w
### optimization full output.x gives the solution array of optimal weights
### ratio or max Sharpe ratio

print(w_Sharpe)
print('return: % .4f'% (ret(r,w_Sharpe)), 'risk: % .4f'% vol(w_Sharpe,covar))
### this is max Sharpe ratio portfolio

print( 1/ ( np.sqrt(np.dot(w_Sharpe,np.dot(w_Sharpe,covar))*252)/r.dot(w_Sharpe) ) )
### Above is optimized objective function -- the max Sharpe ratio.
### It can also be found using print(sharpe(ret(r,w_Sharpe),vol(w_Sharpe,covar)))

[0.    0.774 0.174 0.    0.03  0.022 0.    ]
return:  0.2980 risk:  0.2710
1.0998715406547999
```

```
In [116... # En este fragmento del código, he tenido que agregar la primera línea
```

```
In [117... decovar = covar.copy()

w = w_min ### w is now optimal portfolio weights, sum to 1
num_ports = 100
gap = (np.amax(r) - ret(r,w_min))/num_ports
### np.amax in numpy returns max in the array -- since weights sum to 1, max portf ret is amax(r)
### The above range given by gap starts at ret given by Min Var Portfolio

all_weights = np.zeros((num_ports, len(dfret.columns)))
### all_weights is 20 100 x 8 zero matrix
### Note: len(dfret.columns) is 8 -- there are 8 stocks here
### print(np.shape(all_weights)) gives (100,8) -- same as print(all_weights.shape)

print("*****")
#####
### First note that in Python, a Tuple is a grouping of unnamed, order
### of different types; an array is a collection where elements' value
### are of a single type. ALL_weights is a 2D array of 100 rows each w
### all_weight[0] below is the first 1D sub-array that is 1st row
```

```

### Note: all_weights[0] is [0. 0. 0. 0. 0. 0. 0. 0.], i.e. first row
### Note: all_weights[1] is [0. 0. 0. 0. 0. 0. 0. 0.], i.e. second row
### .....
### Note: all_weights[99] is [0. 0. 0. 0. 0. 0. 0. 0.], i.e. 100th row
### print(all_weights[0].shape) gives (8,) -- a 1-tuple with 8 element
### print(all_weights.shape[0]) gives 100, i.e. dimension of the rows
### print(all_weights.shape[1]) gives 8, i.e. dim of the cols
#####

ret_arr = np.zeros(num_ports) ### this is a 1-tuple of 100 zeros
### Note: print(ret_arr.shape) gives (100,) -- this is a 1D tuple
### print(ret_arr.shape[0]) gives 100, the number of elements in 1D tu
### if we print(ret_arr.shape[1]) - this gives "tuple index out of ran
### If we use instead ret_arr = np.zeros((num_ports,1)), then print(re
### gives (100,1), a dataframe with one column

vol_arr = np.zeros(num_ports)

for i in range(num_ports): ### this means looping from i=0 to 1,2,3,4,
    port_ret = ret(r,w) + i*gap
    double_constraint = LinearConstraint([np.ones(dfret.shape[1]),r],[
### above means left term in first bracket np.ones(dfret.shape[1]) or
### in 2nd,3rd brackets 1,1;
### right term in first bracket r * w >= second terms in 2nd, 3rd brac
### i.e. constraints port wts sum to 1 and portf exp return == port_re

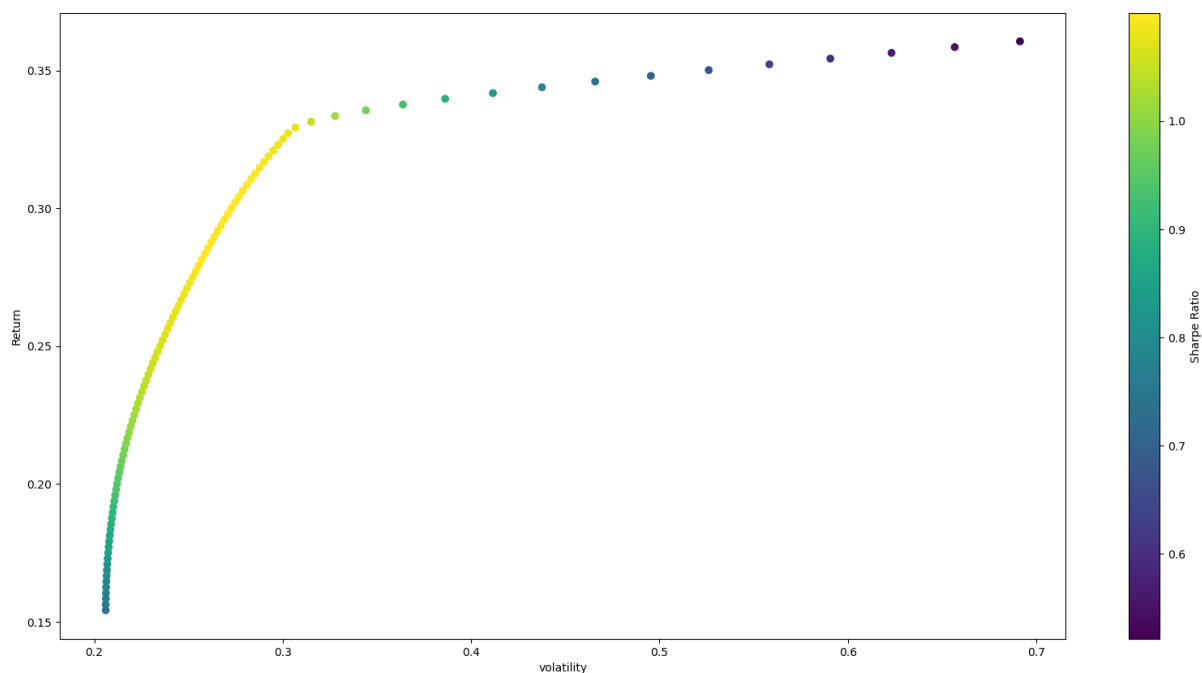
### Create x0: initial guesses for weights.
x0 = w_min
### Define a function for annualized portfolio volatility.
portfvol = lambda w: np.sqrt(np.dot(w,np.dot(w,decovar)))*252)
res = minimize(portfvol,x0,method='trust-constr',constraints = dou
### Above double constraints mean unit vector .dot(w) = 1; r .dot(w) =
all_weights[i,:]=res.x ### 24 x 1 optimal wts at row i, formed usi
ret_arr[i]=port_ret
vol_arr[i]=vol(res.x,decovar)

### Indented paras after "for i..." form the loop

sharpe_arr = ret_arr/vol_arr ### sharpe_arr is 100 x 1 array since it

plt.figure(figsize=(20,10)) ### optimal frontier using training set an
plt.scatter(vol_arr, ret_arr, c=sharpe_arr, cmap='viridis')
### in plt.scatter, c is a scalar or sequence of n numbers to be mapp
### in plt, for sequential plots, 'viridis' gives colors across the 3D
### c= in front of third dimension sharpe_arr gives the colors in that
plt.colorbar(label='Sharpe Ratio')
plt.xlabel('volatility')
plt.ylabel('Return')
plt.show()

```



```
In [118... dfret.shape
```

```
Out[118... (1157, 7)
```

```
In [119... t1=pd.concat([dy,mth,yr], axis=1) #Axis = 1 is important -- aligning c
t1=t1.iloc[1:,:]
print(t1)
```

	day	month	year
1	5.0	1.0	2017.0
2	6.0	1.0	2017.0
3	10.0	1.0	2017.0
4	11.0	1.0	2017.0
5	12.0	1.0	2017.0
...	...	...	...
1153	23.0	12.0	2021.0
1154	27.0	12.0	2021.0
1155	28.0	12.0	2021.0
1156	29.0	12.0	2021.0
1157	30.0	12.0	2021.0

```
[1157 rows x 3 columns]
```

```
In [123... t1.shape
```

```
Out[123... (1157, 3)
```

```
In [121... dfret1=pd.concat([dfret, t1],axis=1)
```

```
In [122... import pandas as pd
dfret1.to_csv('datafiles/ret_portecom.csv')
```

Explicación del código implementado

Lectura de archivo .csv a un dataframe

El código comienza leyendo los retornos diarios de los archivos .csv y realizando algunas transformaciones iniciales:

- Agrega columnas **year**, **month** y **day**, tomando el valor del campo "Date" del dataframe y separando los campos por el separador -.
- Los valores de estas columnas son seteados a **flota**.
- También filtra las filas donde el año no esté entre 2017 y 2021.
- Elimina la columna **Date**.
- Por último, asigna a la variable donde asignó el dataframe únicamente las columnas **Close**, **day**, **month** y **year**.

```
dfAmazon =
pd.read_csv(os.path.join(github_datafiles_directory,
'amzn_us_d.csv'), parse_dates=True)
dfAmazon[["year", "month", "day"]] =
dfAmazon["Date"].str.split("-", expand = True)
cols = dfAmazon.select_dtypes(exclude=['float']).columns
dfAmazon[cols] = dfAmazon[cols].apply(pd.to_numeric,
downcast='float', errors='coerce')
dfAmazon = dfAmazon[(dfAmazon['year']>=2017)&(dfAmazon['year']
<=2021)]
dfAmazon = dfAmazon.drop('Date', axis=1)
dfAmazon = dfAmazon[['Close', 'day', 'month', 'year']]
dfAmazon.head()
```

Este procedimiento se repite para todas las empresas del análisis.

Merge de los retornos diarios de todas las empresas.

El precio ajustado de cierre de las acciones de las distintas empresas son combinados de manera que únicamente todos los precios de cierre ajustados para el mismo día, mes y año sean incluidos. (Por ejemplo, si en alguna fecha el mercado donde una de las empresas hubiera estado cerrado, esa fecha se eliminará del análisis para todas las empresas)

Cálculo de la tasa de retorno compuesta

Se calcula esta tasa que también se puede expresar como $\ln [1 + (P_{t+1} / P_t - 1)]$.

Gráficos de los retornos diarios

Se realizan gráficos de los retornos diarios de todas las empresas seleccionadas.

Cálculo de los retornos diarios acumulados y gráficos

Se calculan los retornos diarios acumulados y luego se grafican.

Portafolios óptimo

Por último, se obtienen los pesos de las acciones de las 7 empresas que forman el portafolios óptimo. Estos pesos son la solución a un problema de minimización donde la función objetivo es el error standard del retorno anual del portafolio. Las medidas clave de retorno medio y desvío standard de este retorno están expresados en valores anualizados y se calculan de la siguiente manera:

- para una acción determinada i , su retorno actualizado es $252 * r_{it}^*$, donde r_{it}^* es la tasa de retorno diaria y es considerada incondicionalmente estacionaria (con la misma media y varianza cada vez). Entonces, la media del retorno anual es $252 * E(r_{it}^*)$ donde $E(.)$ es el operador correspondiente a la Esperanza estadística y $E(r_{it}^*)$ es estimado usando $1/T \sum_{t=1}^T r_{it}^*$ con los retornos diarios r_{it}^* , $t=1,2,...,T$. La media del retorno anualizado es $252 \frac{1}{T} \sum_{t=1}^T r_{it}^*$ donde 252 es el valor aproximado de días donde hay trading de acciones en los Estados Unidos.
- para una acción determinada i , donde $i=1, 2, ..., N$ con un retorno diario r_t en el día t , cada elemento ij de la covarianza σ_{ij} se estima como $1/(T - k) \sum_{t=1}^T (r_{it} - \mu_i)(r_{jt} - \mu_j)$ donde $k = 1$ para $i = j$, y $k = 2$ para $i \neq j$.

La matriz covarianza puede ser denotada en la forma $\text{var}(r_t^*)$. La volatilidad del retorno del portafolios es entonces

$$\sqrt{252x(w^T \text{var}(r_t^*) w)}$$

o bien

```
np.sqrt(np.dot(w,np.dot(w,covar))*252)
```

Para la solución MVP (Mean Volatility Portfolio, o Portafolios de Media y Varianza), cuando tenemos el siguiente código:

```
w_min = res1.x
np.set_printoptions(suppress = True, precision=3)
print(w_min)
print('return: % .4f' % (ret(r,w_min)), 'risk: % .4f' %
vol(w_min,covar))
```

Cuando ejecutamos este código, obtenemos los siguientes resultados:

```
[0.096 0.259 0.25  0.273 0.    0.122 0.    ]
return: 0.1542 risk: 0.2060
```

Esto se debe interpretar como un portafolios óptimo con los siguientes pesos

para las acciones que lo componen:

- 9.6% para Alibaba
- 25.9% para Amazon
- 25% para Ebay
- 27.3% para Rakuten
- 0% para Wayfare
- 12.2% para Zalando
- 0% para JD.com

Y con este portafolio, se obtiene un retorno esperado de 15.42% y un riesgo o desvío estándar de 20.6%

Para el cálculo del ratio de Sharpe (retorno esperado sobre desvío estándar), tenemos este código:

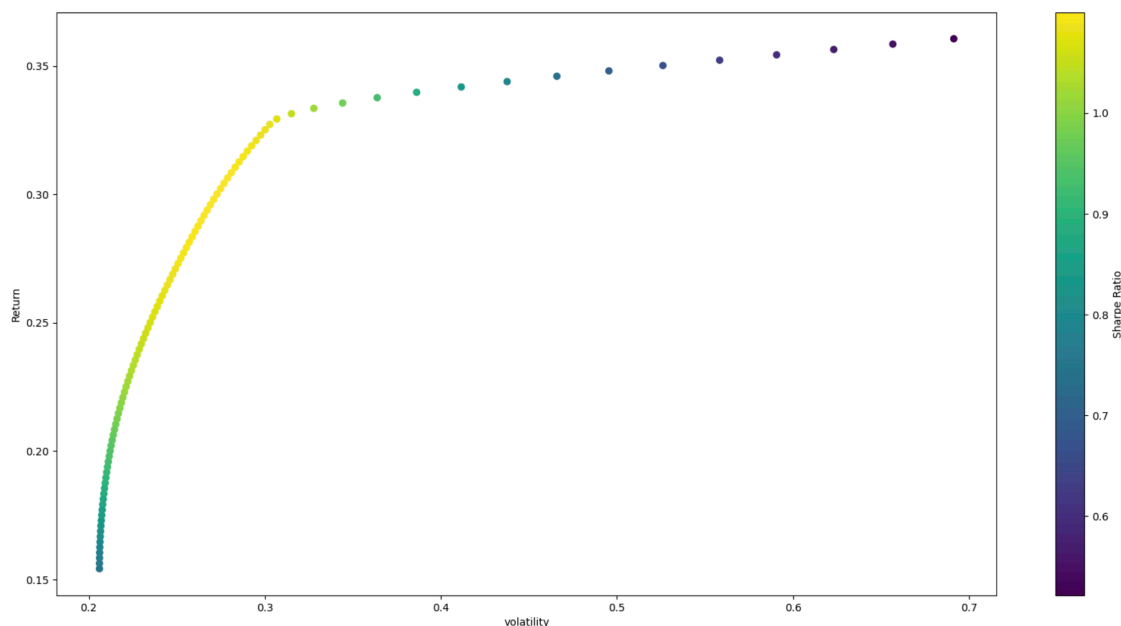
```
invSharpe = lambda w:
np.sqrt(np.dot(w,np.dot(w,covar))*252)/r.dot(w)
res2 = minimize(invSharpe,x0,method='trust-constr',constraints
= linear_constraint,bounds = bounds)
w_Sharpe = res2.x
print(w_Sharpe)
print('return: % .4f'% (ret(r,w_Sharpe)), 'risk: % .4f'%
vol(w_Sharpe,covar))
print( 1/ (
np.sqrt(np.dot(w_Sharpe,np.dot(w_Sharpe,covar))*252)/r.dot(w_Shar
) ) )
```

Cuando ejecutamos este código, obtenemos los siguientes resultados:

```
[0.    0.774 0.174 0.    0.03  0.022 0.    ]
return: 0.2980 risk: 0.2710
1.0998715406547999
```

Esto significa que la solución para el ratio de Sharpe maximizado es 1.0999, mientras que el retorno esperado del portafolios es 29.8% y desvío estándar de 27.10%

Por último, la frontera de portafolios eficiente de acuerdo a los criterios de media y varianza es graficada bajo las restricciones de pesos positivos para todas las acciones del portafolios. Diferentes retornos esperados dan lugar a diferentes volatilidades minimizadas y esto es lo que el gráfico muestra:



Reflexión final

Este artículo presenta la implementación en lenguaje python de un tema central en el análisis financiero como es el del análisis MVP en un portafolios y la determinación de su frontera eficiente. Estas teorías fueron desarrolladas por el Premio Nobel de Economía Harry Markowitz (1990).

El lenguaje de programación **python** junto con las librerías **numpy** y **pandas** son herramientas poderosísimas para la manipulación de datos y los cálculos que se realizan con estos datos, proveyendo también valiosas herramientas para la visualización de los resultados obtenidos.

Declaración del uso de la IA

He utilizado la IA para obtener ayuda sobre cómo usar el lenguaje markdown para las expresiones matemáticas que se incluyen en el Trabajo Práctico.